

Lab 3 Building an OTP Authentication

Server

Demonstration

Successful password + OTP authentication

Client

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab3> python .\auth_client.py
Maximum failed attempts: 3

Enter your username: Alice
Enter your Password:
Attempt 1 → OTP_REQUIRED
Enter your code: 690197
Attempt 1 → ACCESS_GRANTED
Enter message or type EXIT to leave: Hi
Server: OK|Message received from Alice
Enter message or type EXIT to leave: Hello I am Alice
Server: OK|Message received from Alice
Enter message or type EXIT to leave: 
```

As shown in the figure, the client entered the correct username, password, and OTP, and logged in successfully.

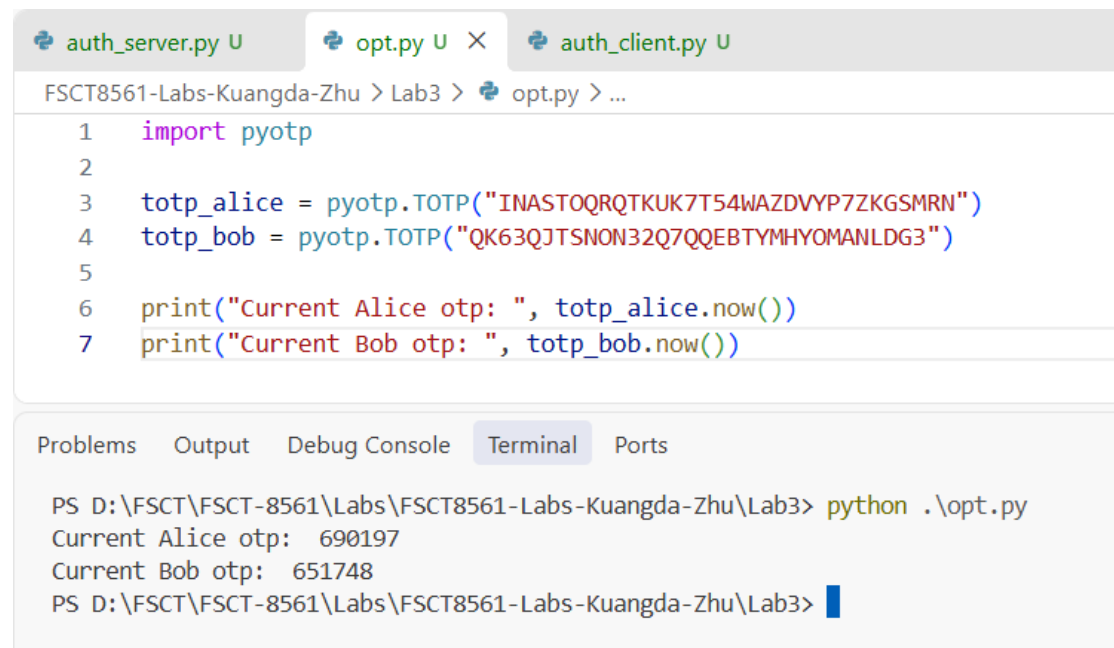
Server

```
Server is waiting for a connection...
Connected by: ('127.0.0.1', 46074)
Received: AUTH|Alice|Cyber123!
Received: OTP|690197
Received: MSG|Hi
Alice says: Hi
Received: MSG|Hello I am Alice
Alice says: Hello I am Alice

```

As shown in the figure, the server successfully received the correct protocol format and can send and receive messages normally.

OPT



The screenshot shows a VS Code editor with three tabs: 'auth_server.py U', 'opt.py U', and 'auth_client.py U'. The 'opt.py' tab is active, displaying the following Python code:

```
1 import pyotp
2
3 totp_alice = pyotp.TOTP("INASTOQRQTKUK7T54WAZDVYP7ZKGSMRN")
4 totp_bob = pyotp.TOTP("QK63QJTSNON32Q7QQEBTYMHYOMANLDG3")
5
6 print("Current Alice otp: ", totp_alice.now())
7 print("Current Bob otp: ", totp_bob.now())
```

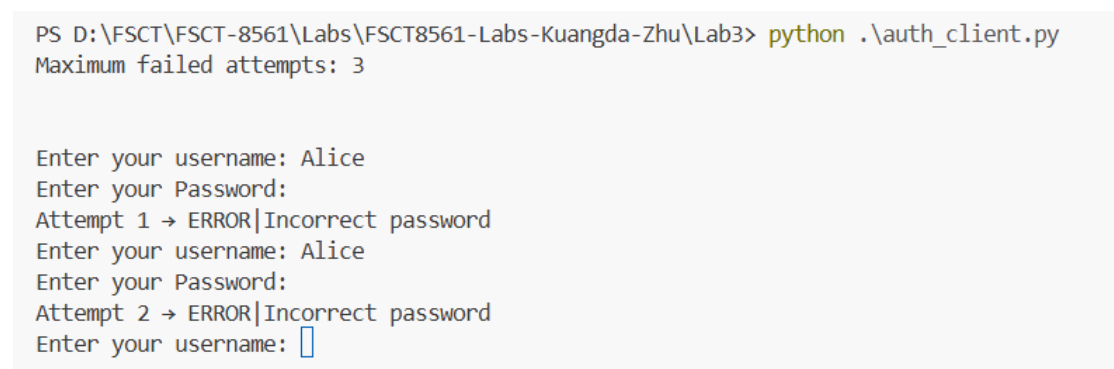
Below the code editor is a terminal window with the following output:

```
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab3> python .\opt.py
Current Alice otp: 690197
Current Bob otp: 651748
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab3>
```

As shown in the figure, I created a Python program to generate time-based OTPs.

Incorrect password

Client



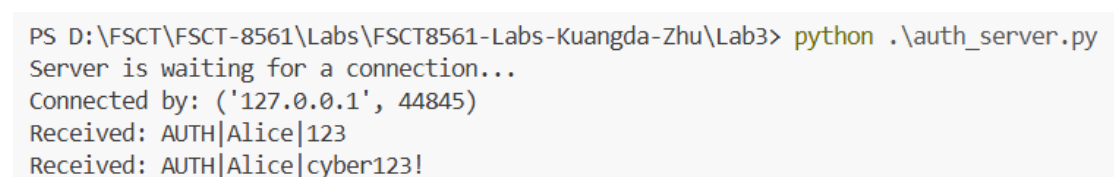
The screenshot shows a terminal window with the following output:

```
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab3> python .\auth_client.py
Maximum failed attempts: 3

Enter your username: Alice
Enter your Password:
Attempt 1 → ERROR|Incorrect password
Enter your username: Alice
Enter your Password:
Attempt 2 → ERROR|Incorrect password
Enter your username: 
```

As shown in the figure, when the correct username and an incorrect password are entered, an ERROR message is returned, but the server continues to run normally.

Server



The screenshot shows a terminal window with the following output:

```
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab3> python .\auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 44845)
Received: AUTH|Alice|123
Received: AUTH|Alice|cyber123!
```

As shown in the figure, the server received the AUTH message, but with an incorrect

password.

Incorrect or expired OTP

Client

```
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab3> python .\auth_client.py
Maximum failed attempts: 3

Enter your username: Alice
Enter your Password:
Attempt 1 → OTP_REQUIRED
Enter your code: 112233
Attempt 1 → ACCESS_DENIED
Enter your code: 690197
Attempt 2 → ACCESS_DENIED
Enter your code: 168154
Attempt 3 → ACCESS_GRANTED
Enter message or type EXIT to leave: █
```

As shown in the figure, entering an incorrect or expired OTP will be rejected until you input the correct time-based OTP.

Server

```
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab3> python .\auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 17115)
Received: AUTH|Alice|Cyber123!
Received: OTP|112233
Received: OTP|690197
Received: OTP|168154
```

As shown in the figure, the server accepted the correct OTP protocol.

OPT

```
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab3> python .\opt.py
Current Alice otp: 690197
Current Bob otp: 651748
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab3> python .\opt.py
Current Alice otp: 317908
Current Bob otp: 822783
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab3> python .\opt.py
Current Alice otp: 168154
Current Bob otp: 096458
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab3> █
```

As shown in the figure, the OTP generated by the program changes correctly over time.

Invalid username

Client

```
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab3> python .\auth_client.py
Maximum failed attempts: 3
```

```
Enter your username: 123
Enter your Password:
Attempt 1 -> ERROR|User not exists
Enter your username: alice
Enter your Password:
```

Server

```
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab3> python .\auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 51371)
Received: AUTH|123|321
Received: AUTH|alice|1321
```

As shown in the figure, when a username not in the database is entered, an ERROR message is returned and prompts the user to retry, and the system is case-sensitive.

OTP-before-password rejection

Client

```
343  if __name__ == "__main__":
344      if test_mode == 7:
345          robust_test_7()
346      if test_mode == 6:
347          robust_test_6()
348      else:
349          main()
```

Problems Output Debug Console Terminal Ports

```
PS D:\FSCT\FSCT-8561\Labs\F SCT8561-Labs-Kuangda-Zhu\Lab3> python .\auth_client.py
Enter your username: 123
Enter your Password:
Server: ERROR|User not exists
Enter your code: 114455
Server: ERROR|Please input the correct username and password first before OTP
Enter your code: 
```

As shown in the figure, we created two functions to test robustness. After entering a non-existent username and password to bypass password authentication, the system returns an error message prompting us to enter the correct username and password first.

Server

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab3> python .\auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 38718)
Received: AUTH|123|321
Received: OTP|114455
█
```

As shown in the figure, we can see on the server side that the OTP protocol was received without proper authentication.

Malformed input handling

Client

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab3> python .\auth_client.py
Enter your username: Alice
Enter your Password:
Server: ERROR|Invalid username and password format
Enter your username: █
```

As shown in the figure, when the format of the username and password is incorrect, an error prompt is returned.

Server

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab3> python .\auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 12062)
Received: AUTH|Alice
█
```

As shown in the figure, the system received a message containing only the username without a password.

Repeated failed attempts

Client

```
PS D:\FSCT\FSC8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab3> python .\auth_client.py
Maximum failed attempts: 3

Enter your username: 123
Enter your Password:
Attempt 1 → ERROR|User not exists
Enter your username: Alice
Enter your Password:
Attempt 2 → ERROR|Incorrect password
Enter your username: Alice
Enter your Password:
Attempt 3 → OTP_REQUIRED
Enter your code: 113344
Attempt 3 → ACCESS_DENIED

Too many failed attempts.
Authentication blocked.
Disconnected
```

Server

```
PS D:\FSCT\FSC8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab3> python .\auth_server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 50657)
Received: AUTH|123|321
Received: AUTH|Alice|123ad
Received: AUTH|Alice|Cyber123!
Received: OTP|113344
Client disconnected unexpectedly
Server closed
PS D:\FSCT\FSC8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab3> █
```

As shown in the figures, the system allows a maximum of 3 failed login attempts before disconnecting. The three errors are different, which are an invalid username, an incorrect password, and an incorrect OTP respectively.

Security Reflection questions

1. Why should passwords not be stored in plaintext?

To prevent attackers from reading user credentials directly if the database is compromised.

2. What is the purpose of hashing a password before storage?

To store irreversible cryptographic data that cannot be reverse-engineered, while

still allowing verification.

3. Why is plain SHA-256 alone insufficient for production password storage?

The high calculation speed makes it vulnerable to GPU-based brute-force and dictionary attacks.

4. What additional protection does TOTP provide beyond a password?

It implements multi-factor authentication (MFA) via time-dependent dynamic tokens.

5. How can TOTP reduce the usefulness of a captured one-time code after it expires?

The short validity window ensures intercepted codes quickly become obsolete.

6. What happens when the user and server clocks are significantly out of sync?

Time drift causes verification failures due to token generation mismatches.

7. What happens if an attacker obtains the user's TOTP secret?

The attacker can persistently forge valid authentication codes.

8. Why should repeated failed login attempts be limited or monitored?

To mitigate automated brute-force and credential stuffing threats.

9. Does `getpass()` protect the password while it travels across the network? Explain.

No. It only hides terminal echoing; network transmission remains unprotected.

10. What additional security would be required before deploying this authentication system in the real world?

TLS/HTTPS transport encryption.

Investigation

1. Why does TOTP depend on synchronized time?

Both the client and server use time to generate the code, so their clocks must match.

2. What happens when a code is submitted after its validity window?

The server rejects the code and authentication fails.

3. Why might a small amount of clock tolerance sometimes be necessary?

To allow for minor network delays or time drift between devices.

4. What security trade-off could occur if the server accepts too many neighboring time windows?

It increases the risk of replay attacks, making old codes work longer.

Challenge Questions

1. Where should the failed-attempt counter be stored?

It is tracked in the client's main loop where both password and OTP failures count toward the limit.

2. Should password failures and OTP failures count the same way?

Yes, they generally count the same way because both represent an invalid authentication attempt that could indicate a brute-force attack.

3. Why can permanent account lockout itself create a denial-of-service problem?

Because malicious attackers can intentionally trigger failed attempts to lock out legitimate users, denying them access to their accounts (Denial of Service).

4. How would a production system improve on this simple approach?

IP-based.

Security Analysis

In addition to the password, our authentication system also utilizes a unique TOTP secret for each user. This secret is generated using Python's built-in library functions and is stored directly in the database alongside the user credentials. For password protection, user passwords are not stored in plaintext; instead, they are protected directly within the Python code and database using cryptographic hashing. This ensures that even if malicious attackers successfully compromise the backend source code or the database files, they cannot read the original user passwords directly. To enhance defense mechanisms, TOTP introduces a time-dependent dynamic code layer on top of standard static password authentication. This successfully combines "something you know" (the password) with "something you have" (the authenticator device), achieving

multi-factor authentication (MFA) to significantly improve security. Using a password combined with TOTP effectively helps mitigate standard credential-based threats, such as static password sniffing, dictionary attacks, and unauthorized account access even if a static password is leaked. Furthermore, implementing a strict limit on repeated failed authentication attempts helps prevent automated brute-force attacks. However, remaining attack vectors still persist. For instance, if an attacker steals the TOTP secret directly from the storage, they can continuously generate valid codes and bypass authentication using replay attacks, even without knowing the user's password. Additionally, transmitting all credentials over a raw TCP connection means data is sent in plaintext across the network, making it highly vulnerable to packet sniffing and man-in-the-middle attacks. Finally, while SHA-256 is designed as a high-speed cryptographic hash function for data integrity checking, this high speed is a limitation for password storage because it allows attackers to use modern GPUs to crack password hashes easily via high-speed brute-force attacks.

Therefore, for a real-world production deployment, we must adopt robust improvements such as implementing TLS transport encryption to secure network traffic and replacing raw hashes with salted, slow key-derivation functions.

Github

<https://github.com/bnwzc/FSCT8561-Labs-Kuangda-Zhu>